DB08

SQL

IBM Research에 의해 개발된 **Special-purpose declaretive programming language**

Data Definition Language (DDL)

> **Database의 구조와 관련 있는 Database, table, index에 대한 명령어를 제공**

CREATE SCHEMA

SQL Schema를 생성한다.

- **SQL Schema**: table 집합 + table과 관련된 Constructs
  - 같은 데이터베이스 어플리케이션에서 사용되는 것을 묶는다.
  - 큰 데이터베이스가 존재하고 내부에 여러 Schema가 존재한다.
  - 각 Schema안에 Table, view 등이 존재한다.

- **CREATE SCHEMA (schema name)**으로 사용된다.
  - Identifier**:** schema name

- **EX)** ‘Jsmith’가 소유하는 COMPANY schema를 생성
  - **CREATE SCHEMA** COMPANY **AUTHORIZATION** ‘Jsmith’;
  - ‘Jsmith’는 **COMPANY schema에 대해 최상위 권한**을 갖게 된다.

CREATE TABLE

새로운 table (Relation)을 형성한다.

- Table name, Attribute list of <attribute_name, data_type>, Initial constraints

```
CREATE TABLE DEPARTMENT
(Dname          VARCHAR(10)  NOT NULL,
 Dnumber        INT          NOT NULL,
 Mgr_ssn        CHAR(9)      NOT NULL,
 Mgr_start_date DATE,
PRYMARY KEY (Dnumber),
UNIQUE (Dname)
FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
```

Attributes

- **Data types**: INT, FLOAT, DECIMAL, CHAR(n), VARCHAR(n), DATE
  - VARCHAR(n): 최대 길이가 n이고 길이 변화 가능
  - DECIMAL(p, s)
    - p: 전체 자릿수, s: 소수점 이하 자릿수
    - 전체 자릿수보다 작아도 된다. 그러나 소수점 이하 자릿수 제한을 초과할 수는 없다.
- **NOT NULL** attribute constraints
- **DEFUALT** Clause

Initial constraints

- **PRIMARY KEY**
  - 각 Tuple을 구별할 수 있도록 하는 하나 이상의 Attribute를 정의
- **UNIQUE** clause
  - PK가 아닌 Candidate key 중 하나.
  - 해당 컬럼에 대해 중복되는 값이 존재하면 안된다.
- **FOREIGN KEY** clause
  - **Referential intergrity constraints**를 위해 Foreign key를 특정
    - FK는 반드시 다른 테이블에 존재하는 PK 또는 Unique key의 값이나 NULL을 가져야 한다.
  - Referential intergrity constraints가 깨질 수 있는 상황에 취할 수 있는 **Referential triggered action**도 정의할 수 있다.
    - Trigger: ON DELETE, ON UPDATE
    - Actions: SET NULL, CASCADE, SET DEFAULT
      - CASCADE는 참조 대상 Tuple이 삭제되거나 수정되면 동일하게 삭제하거나 수정하는 것이다.
  - **Referential intergirity constraints 때문에 CREATE TABLE을 하는 순서도 중요**하다.
    - 만약 DEPARTMENT 테이블의 Mgr_ssn이 EMPLOYEE Table의 Ssn을 참조한다면 EMPLOTYEE 테이블이 생성되어야 **FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)**을 사용할 수 있다.
    - 만약 DEPARTMENT 테이블을 먼저 생성하였다면, EMPLOYEE 테이블을 생성한 이후에 **ADD CONSTRAINT**를 통해 추가하도록 한다.
      - **ALTER TABLE** table_name **ADD CONSTRAINT** constraint_name **FOREIGN KEY** (Mgr_ssn) **REFERENCES** EMPLOYEE(Ssn)
- **CONSTRAINT** constraint_name **[CONSTRAINT 명령어]**를 통해 Constraint의 이름을 지정할 수 있다.
  - 이렇게 하는 이유는 나중에 Constraint를 관리할 때 편리하도록 하기 위함이다.

- **ALTER TABLE** EMPLOYEE **DROP CONSTRAINT** constraint_name 등에서 constraint_name이 사용된다.
- 아래와 같이 Constraint를 다룰 수 있다.

```
CONSTRAINT EMPSUPERFK
    FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
                    ON DELETE SET NULL          ON UPDATE CASCADE,
CONSTRAINT EMPDEPTFK
    FOREIGN KEY(Dno) REFERENCES DEPARTMENT(Dnumber)
                    ON DELETE SET DEFAULT       ON UPDATE CASCADE);
```

DROP SCHEMA

> **더 이상 필요하지 않은 SQL Schema를 삭제한다.**

DROP SCHEMA schema_name **CASCADE / RESTRICT;**

- **CASCADE:** Schema 내의 Element까지 전부 제거
- **RESTRICT**: Schema 내의 Element가 존재하지 않는 경우에만 삭제할 수 있도록 함
- Element: Table, View ...

DROP TABLE

> **Table 정의와 Tuple들을 모두 제거**

DROP TABLE table_name **CASCADE / RESTRICT;**

- **CASCADE:** 테이블과 해당 테이블을 참조하는 것을 모두 제거
- **RESTRICT**: 해당 테이블을 참조하는 것이 없는 경우에만 제거 가능

ALTER TABLE

Schema 관점에서 Table 구조를 변경하는 명령어

- Schema 속 Table 자체의 정의를 수정

기능

- Column (Attribute) 추가/제거
  - **ADD COLUMN / DROP COLUMN**
- Column 정의 변경
  - **ALTER COLUMN**
- Table constraint 추가 / 제거
  - **ADD CONSTRAINT / DROP CONSTRAINT**

예시

- **ALTER TABLE** COMPANY.EMPLOYEE
  ADD COLUMN Job **VARCHAR**(12);

- **ALTER TABLE** COMPANY.EMPLOYEE
  DROP COLUMN Address **CASCADE**;
  - 해당 Column을 참조하는 다른 것들까지 함께 제거
  - **RESTRICT: 참조하는 것이 있으면 제거되지 않는다.**

- **ALTER TABLE** COMPANY.EMPLOYEE
  ALTER COLUMN Mgr_ssn **DROP DEFAULT**;
  - **SET NOT NULL**, **DROP NOT NULL도 가능하다.**

- **ALTER TABLE** COMPANY.EMPLOYEE
  ALTER COLUMN Mgr_ssn **SET DEFAULT** '333445555';

- **ALTER TABLE** COMPANY.EMPLOYEE
  DROP CONSTRAINT EMPSUPERFK;
  - EMPSUPERFK: Contraint_name

Data Manipulate Language (DML)

> **Tuple에 대한 명령어를 제공**

SELECT

Database에서 **정보를 가져오기 위해 사용**하는 가장 기본적인 명령어

- 기본이 되는 Retrieval query
- Relational algebra의 SELECT와 다르다.
- **Relational algebra의 SELECT, JOIN, PROJECT를 하나로 합친 것**이다.
- DML의 기본이 된다.

Relational model에서와 다르게 같은 **테이블에서 완전히 동일한 Tuple을 허용**한다.

- 중복을 허용하지 않도록 하기 위해선 **DISTINCT clause**을 사용할 수 있다.

SELECT <attribute list>
FROM <table list>
WHERE <condition> : Boolean expression

SELECT → 3. 해당하는 Attribute만을 보여줘
FROM → 1. 이 Table의 여러 튜플 중에서
WHERE → 2. 이 조건을 만족하는 Tuple들의

FROM 절에 여러 Table이 오는 경우 **해당 Table들의 CARTESIAN PRODUCT로 생성된 Table**을 생각하면 된다.

- 이 경우 WHERE 절의 Condition에 **JOIN CONDITION**이 포함된 경우가 많다.
- SELECT문에서 **SELECT** attribute1 **JOIN** attribute2 **ON** join_condition 으로 사용할 수도 있다.

SQL은 **Declaritive langauge**이기 때문에 FROM 절에 나열된 Table의 순서나 WHERE 절에 나열된 조건의 순서는 결과에 미치지 않는다.

- DBMS가 어차피 알아서 최적화한다.

그러나 **SELECT 내의 Attribute 순서는 결과에 직접적 영향**을 끼친다.

- 결과로 된 Table의 attribute 순서가 달라지기 때문이다.

Ambiguous attribute name

SELECT 문 내에서 여러 Table이 사용될 때, 각 **Table의 같은 이름을 갖는 Attribute가 존재하는 경우 Attribute_name만 사용할 경우 애매해질 수 있다.**

- 이 경우 반드시 `Table_name.attribute_name`을 사용해야 한다.

Table Aliases

FROM EMPLOYEE **AS** A, DEPARTMENT **AS** D와 같이 별칭을 설정할 수 있다.

- 특히, 같은 테이블을 복제하여 FROM 절에 사용하는 경우 반드시 Aliasing을 해야 한다.
    - **Recursive relation**이 이 경우에 속한다.

FROM EMPLOYEE **AS** E(FN, MI, LN, SSN, BD, ADDR, SEX, SAL, SSSN, DNO)

- Table alias와 Attribute alias가 동시에 설정된다.

위 모두 **해당 Query 내에서만 Table, attribute의 이름을 다르게 사용할 수 있는 것이지 실제로 DB에서 변경되는 것은 아니다.**

Aliasing을 하는 경우, **해당 Query에서는 반드시 Aliasing를 사용**해야 한다.

Missing WHERE clause

이 경우 **조건이 없기** 때문에 모든 Tuple이 반환된다. 결국 두 Table의 **CARTESIAN PRODUCT를 하는 것과 동일**하다.

Asterisk

모든 Attribute를 확인하고 싶을 때, Select에 모든 Attribute name을 쓰는 대신 ***을 사용**하면 된다.

DISTINCT

SQL SELECT의 결과는 중복된 Tuple을 제거하지 않는다. **중복된 Tuple을 제거하고 싶을 때 사용하는 Clause**

- 중복을 허용하는 집합 → Set (진정한 집합)

SELECT DISTINCT <attribute list>로 사용한다.

사실 이전 SELECT 문은 모두 **SELECT ALL** <attribute list> 상태인데, **ALL이 기본값이라 생략한 것이다.**

- 중복을 허용하는 이유는 합, 평균 등을 구할 때 생략되면 안되기 때문이다.

SET OPERATION

UNION / EXCEPT / INTERSECTION

Set operation의 결과는 **Set**이다. 따라서 **중복을 허용하지 않는다.**

- Set operation의 **입력이 중복이 허용되는 집합이어도 결과는 중복이 제거된다.**

```
(SELECT   DISTINCT Pnumber
 FROM     PROJECT, DEPARTMENT, EMPLOYEE
 WHERE    Dnum=Dnumber AND Mgr_ssn=Ssn
          AND Lname='Smith' )
 UNION
( SELECT  DISTINCT Pnumber
 FROM     PROJECT, WORKS_ON, EMPLOYEE
 WHERE    Pnumber=Pno AND Essn=Ssn
          AND Lname='Smith' );
```

- 이 경우에서 SET Operation인 **UNION**이 중복을 제거하기 때문에 **DISTINCT**는 사용하지 않아도 무방하다.

Tip

SQL에서 **DROP / ALTER는 Schema를 다루고, DELETE / UPDATE는 Tuple을 다룬다.**